



多媒體技術與應用

Spring, 2026.

Instructor : Yen-Lin Chen(陳彥霖), PH.D.
Distinguished Professor
Dept. Computer Science and Information Engineering
National Taipei University of Technology



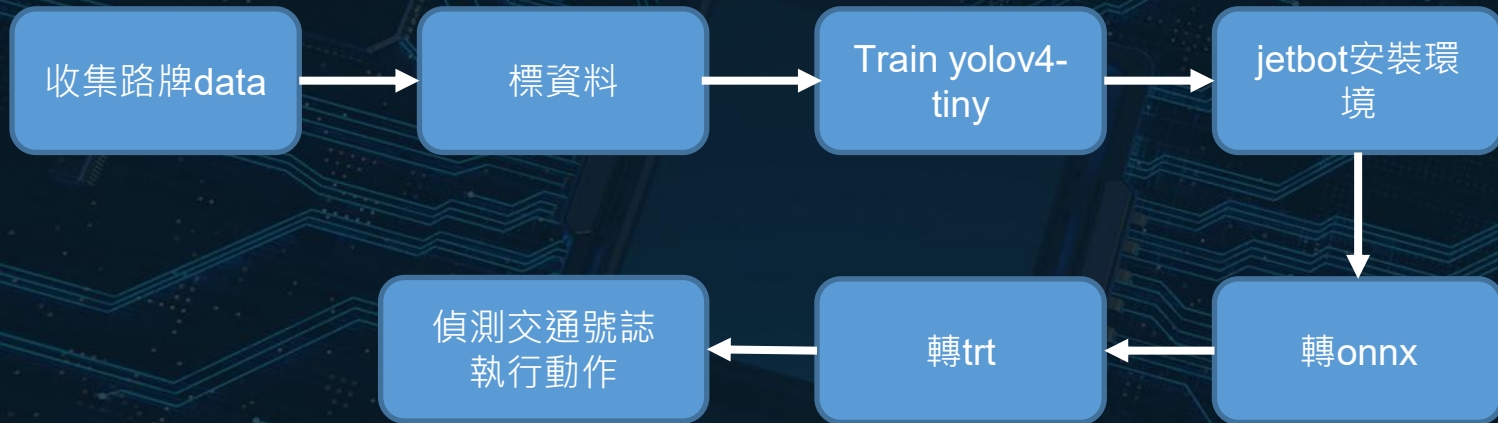
路牌辨識

- 本專案將結合先前所深入研習的 YOLO 模型技術，從資料集的採集到模型推論優化，逐步建構出一套專屬於自走車的影像感知架構，使其不僅能即時辨識各類路牌標誌，更能展現出自動駕駛研發中關鍵的環境理解與反應能力。





流程圖





收集路牌資料

下載 jetcam (使用USB攝影機適用)

1. 在Ubuntu桌面按右鍵
2. git clone <https://github.com/NVIDIA-AI-IOT/jetcam>
3. cd jetcam
4. python3 setup.py install

```
jetbot@jetson-4-3: ~/jetcam
jetbot@jetson-4-3:~$ git clone https://github.com/NVIDIA-AI-IOT/jetcam
Cloning into 'jetcam'...
remote: Enumerating objects: 191, done.
remote: Total 191 (delta 0), reused 0 (delta 0), pack-reused 191
Receiving objects: 100% (191/191), 41.34 KiB | 529.00 KiB/s, done.
Resolving deltas: 100% (78/78), done.
jetbot@jetson-4-3:~/jetcam$ cd jetcam/
jetbot@jetson-4-3:~/jetcam$ python3 setup.py install
running install
running bdist_egg
running egg_info
creating jetcam.egg-info
writing jetcam.egg-info/PKG-INFO
writing dependency_links to jetcam.egg-info/dependency_links.txt
writing top-level names to jetcam.egg-info/top_level.txt
writing manifest file 'jetcam.egg-info/SOURCES.txt'
reading manifest file 'jetcam.egg-info/SOURCES.txt'
writing manifest file 'jetcam.egg-info/SOURCES.txt'
installing library code to build/libdist/linux-aarch64/egg
running install_lib
running build_py
creating build
creating build/lib
creating build/lib/jetcam
```




收集路牌資料

- 若遇到權限問題(Permission denied) , 改成 **sudo python3 setup.py install**

```
jetbot@jetson-4-3:~/jetcam$ python3 setup.py install
running install
error: can't create or remove files in install directory

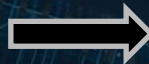
The following error occurred while trying to add or remove files in the
installation directory:

[Errno 13] Permission denied: '/usr/local/lib/python3.6/dist-packages/te
asy-install-11159.write-test'

The installation directory you specified (via --install-dir, --prefix, or
the distutils default setting) was:

/usr/local/lib/python3.6/dist-packages/

Perhaps your account does not have write access to this directory?  If the
installation directory is a system-owned directory, you may need to sign in
as the administrator or "root" account.  If you do not have administrative
access to this machine, you may wish to choose a different installation
directory, preferably one that is listed in your PYTHONPATH environment
variable.
```



```
jetbot@jetson-4-3:~/jetcam$ sudo python3 setup.py install
[sudo] password for jetbot:
running install
running bdist_egg
running egg_info
creating jetcam.egg-info
writing jetcam.egg-info/PKG-INFO
writing dependency_links to jetcam.egg-info/dependency_links.txt
writing top-level names to jetcam.egg-info/top_level.txt
writing manifest file 'jetcam.egg-info/SOURCES.txt'
reading manifest file 'jetcam.egg-info/SOURCES.txt'
writing manifest file 'jetcam.egg-info/SOURCES.txt'
installing library code to build/bdist.linux-aarch64/egg
```



收集路牌資料

- 開啟分頁，並輸入ip:8888，ip是對應到車車上的wlan0





收集路牌資料

- 之後會跳出一個 jupyter 頁面，輸入密碼 **jetbot**
- (若你的電腦連不上，有可能你的電腦跟車車不是在同個網路下)

jupyter

Password:

Log in

收集路牌資料

- 進去之後，會看到的畫面如圖所示，點入 **road_following** 資料夾，進入 road_following 資料夾後會看到三個預設的檔案，分別是 **data_collection.ipynb**、**live_demo.ipynb**、**train_model.ipynb**

/ Notebooks /	
Name	Last Modified
basic_motion	3 years ago
collision_avoidance	3 years ago
object_following	3 years ago
road_following	3 years ago
teleoperation	3 years ago

/ Notebooks / road_following /	
Name	Last Modified
data_collection.ipynb	3 years ago
live_demo.ipynb	3 years ago
train_model.ipynb	3 years ago



收集路牌資料

- 可以選擇用data_collection.ipynb來收集資料(參考資料：<https://blog.cavedu.com/2020/11/05/rk-jetbot/>)
- 滑鼠點擊左側相機影像可以進行拍攝

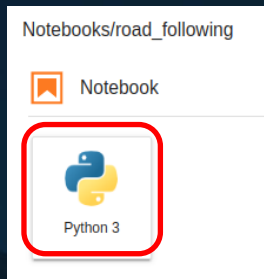
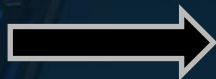
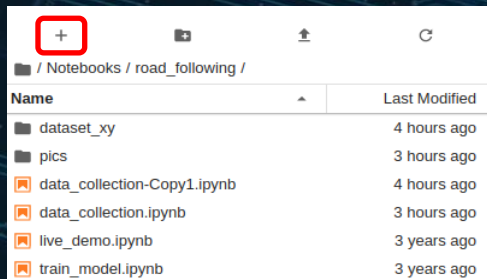




收集路牌資料

也可以使用OpenCV的方式蒐集資料。

首先點選左上角的+號後，點選Notebook下的Python3



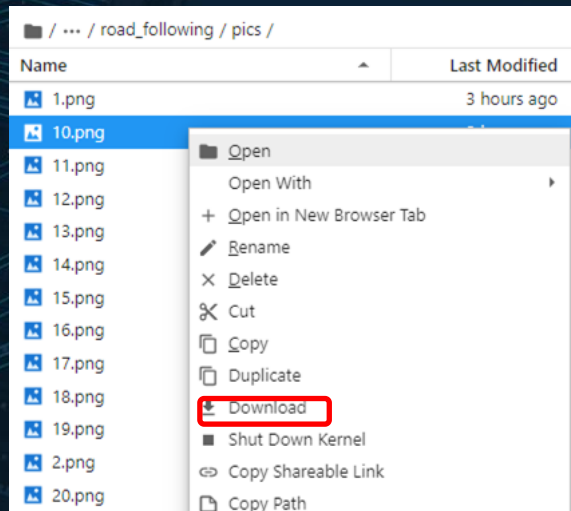
可參考右側的程式碼進行更改，
加入迴圈或time.sleep()等方式，
控制拍照的間隔集張數

```
# 匯入函式庫
import cv2
# 設定從哪顆鏡頭讀取影像，本範例中為video0
webcam = cv2.VideoCapture(0)
# 讀取影像
return_value, image = webcam.read()
# 儲存名為1.png的照片
cv2.imwrite("pics/1.png", image)
# 刪除webcam，養成不佔用資源的好習慣
del(webcam)
```




收集路牌資料

- 拍好的照片記得Download備份





LabelImg標資料

- 如同 Projet04 的做法，先進到labelImg.exe存放的資料夾內並點選data資料夾，修改

```
defined_classes.txt
```
- 分別是：

```
stop
```

、

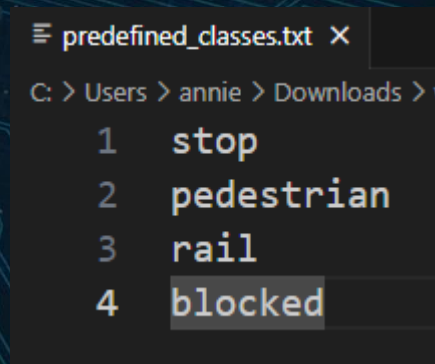
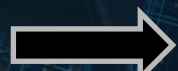
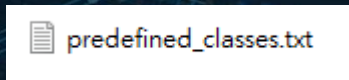
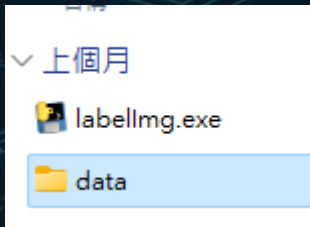
```
pedestrian
```

、

```
rail
```

、

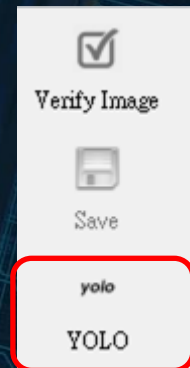
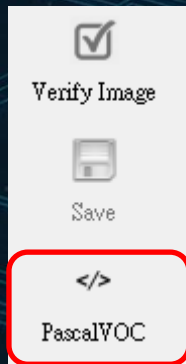
```
blocked
```





LabelImg標資料

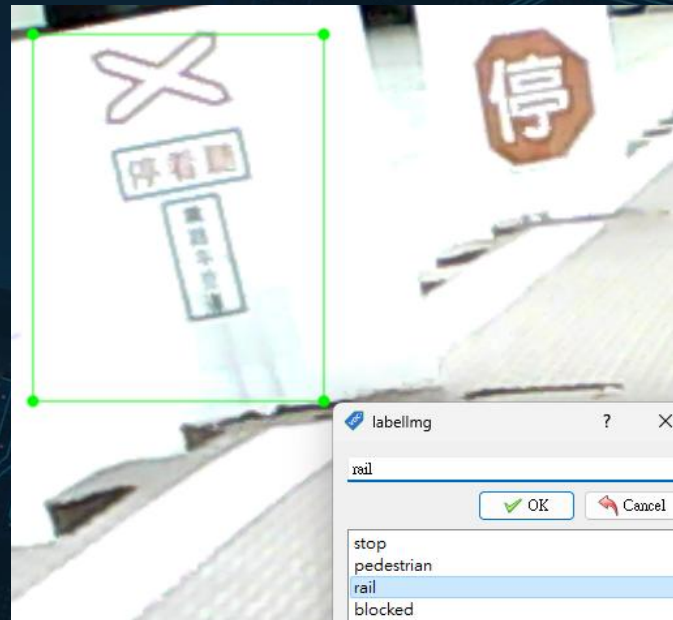
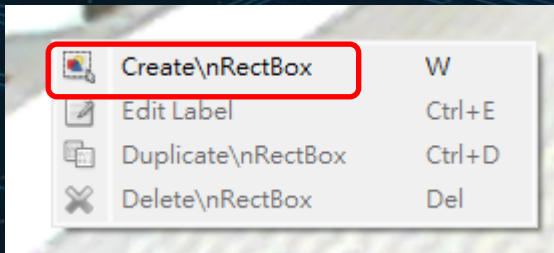
- 把側邊欄的資料格式從PascalVOC換為YOLO(點一下就可以了)





Labellmg

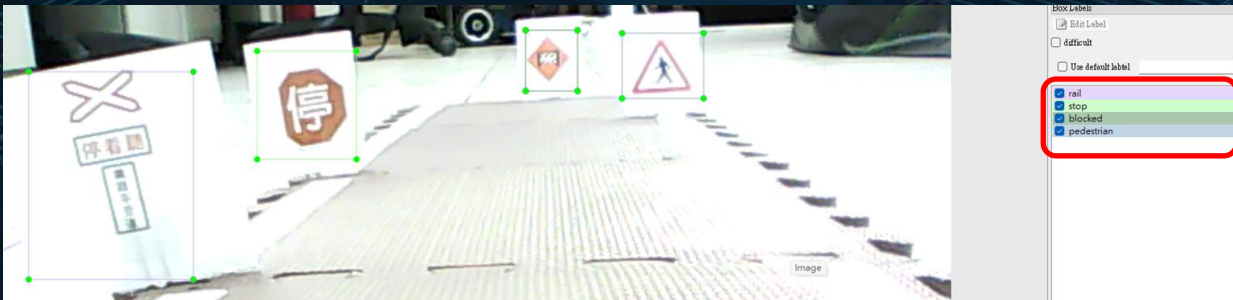
- 在圖上右鍵，點選Create\nRectBox，選擇相對應的標籤





LabelImg標資料

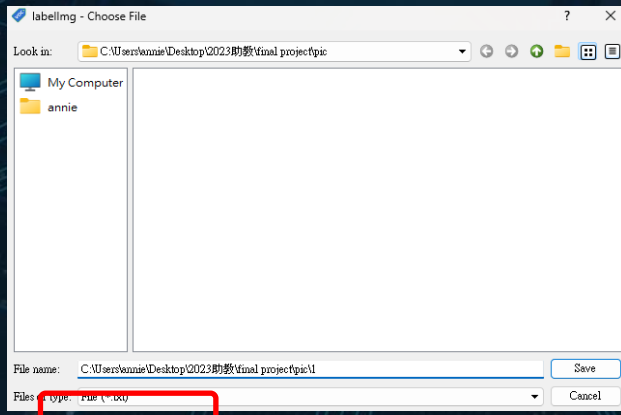
- 標好會顯示在右邊，這張圖都標完，記得按 Ctrl S 進行存檔
- (每一張圖都要存一次!!!很重要)





LabelImg標資料

- 一張圖片會對應到一個txt檔，圖片與他的txt檔名要一樣，所以通常File name 跟type 都不會做更動。
- 若你有20張label好的圖，那你應該要有20個txt檔。





Train yolov4-tiny

- Google Colab連結自己的Google雲端硬碟空間
 - 由於Colab的空間是一個臨時的工作空間，在建立當初是沒有任何檔案的，此時可將Colab連結自己的Google雲端硬碟空間，以利將事先準備好的obj.names、obj.data等檔案複製到Colab的工作空間，或是將訓練好的.weight權重檔案複製回雲端。
 - 我們在自己的運算硬碟上執行，文件就不會消失



建立Project

The screenshot shows the Google Drive web interface. On the left, the sidebar includes navigation options like '新增' (New), '首頁' (Home), '活動' (Activity), '工作區' (Workspace), and '我的雲端硬碟' (My Drive). The main area displays '我的雲端硬碟' (My Drive) with a list of files and folders. A folder named 'FinalProject' is selected. A context menu is open over the 'FinalProject' folder, showing options like '新資料夾' (New folder), '檔案上傳' (Upload file), '資料夾上傳' (Upload folder), and a list of Google services. 'Google Colaboratory' is highlighted in the list.

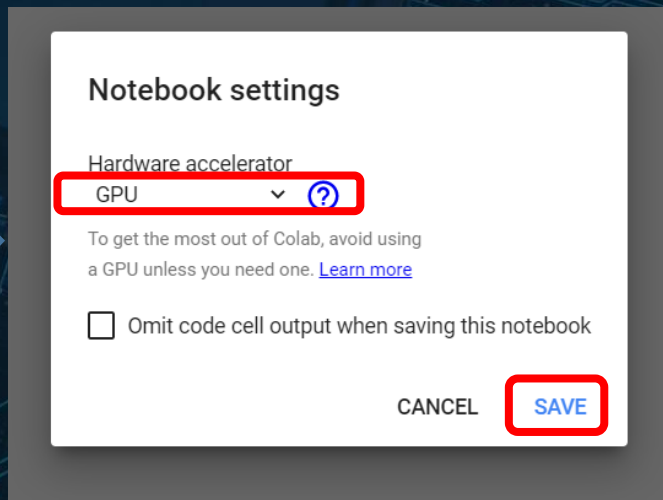
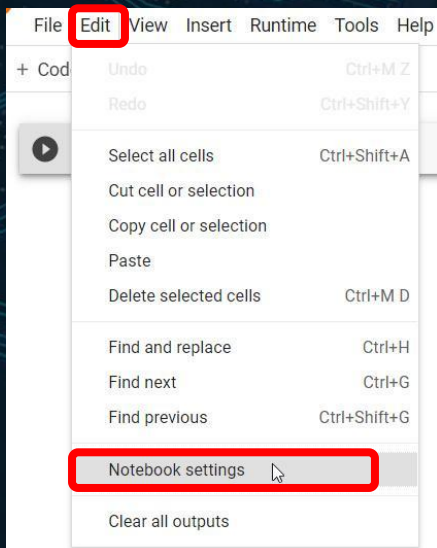
名稱	擁有者	修改日期	檔案大小
Colab Notebooks	我	10月30日 我	—
FinalProject	我	下午2:34 我	—
42_testing_image.zip			
42_training_image.zip			
42_training_label.zip			
50.zip			
aortic_valve_colab.yaml			

1.請先在自己的雲端建立專案資料夾

2.進入資料夾後開啟colab

Train yolov4-tiny

- 開啟 Google Colab 然後啟用免費的GPU
- 網址：<https://colab.research.google.com/>





使用 Google Colab 雲端運算平台

- Google Colab 連結自己的 Google 雲端硬碟空間
 - 由於 Colab 的空間是一個臨時的工作空間，在建立當初是沒有任何檔案的，此時可將 Colab 連結自己的 Google 雲端硬碟空間，以利將事先準備好的 obj.names、obj.data 等檔案複製到 Colab 的工作空間，或是將訓練好的.weight 權重檔案複製回雲端。
 - 我們在自己的運算硬碟上執行，文件就不會消失。

```
# 掛載自己的雲端硬碟
from google.colab import drive
drive.mount('/content/drive')

# 建立 MyDrive 的捷徑，方便存取
!ln -s /content/drive/MyDrive /mydrive

... Mounted at /content/drive
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks

Connect to Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
!ln -s /content/drive/MyDrive /mydrive
```


Train yolov4-tiny

掛載完自己的雲端硬碟之後，應該可以在右側的檔案找到自己雲端硬碟的位置
Drive > MyDrive，新增完之後clone darknet

```
%cd /mydrive/FinalProject/  
!git clone https://github.com/AlexeyAB/darknet
```



```
%cd /mydrive/FinalProject/  
!git clone https://github.com/AlexeyAB/darknet  
  
... /content/drive/MyDrive/FinalProject  
Cloning into 'darknet'...  
remote: Enumerating objects: 15909, done.  
remote: Counting objects: 100% (6/6), done.  
remote: Compressing objects: 100% (4/4), done.  
remote: Total 15909 (delta 2), reused 2 (delta 2), pack-reused 15903 (from 2)  
Receiving objects: 100% (15909/15909), 14.43 MiB | 8.27 MiB/s, done.  
Resolving deltas: 100% (10710/10710), done.
```



Train yolov4-tiny

- 下載完成後，我們需要修改 Makefile 裡面的四個參數，分別是：
 - GPU=0 要改成 GPU=1（開啟GPU加速）
 - OPENCV=0 要改成 OPENCV=1（用來讀取影像、影片、畫框等功能）
 - CUDNN=0 要改成 CUDNN=1（用於加速tensorflow、pytorch等深度學習框架）
 - CUDNN_HALF=0 要改成 CUDNN_HALF=1（建構tensor核心，加速偵測物件）

```
%cd /mydrive/FinalProject/darknet
!sed -i "s/GPU=0/GPU=1/" Makefile
!sed -i "s/OPENCV=0/OPENCV=1/" Makefile
!sed -i "s/CUDNN=0/CUDNN=1/" Makefile
!sed -i "s/CUDNN_HALF=0/CUDNN_HALF=1/" Makefile
```

```
... /content/drive/MyDrive/FinalProject/darknet
```

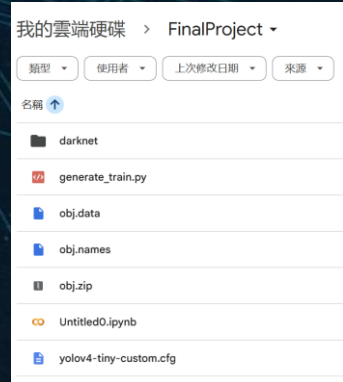
```
%cd /mydrive/FinalProject/darknet
!sed -i "s/GPU=0/GPU=1/" Makefile
!sed -i "s/OPENCV=0/OPENCV=1/" Makefile
!sed -i "s/CUDNN=0/CUDNN=1/" Makefile
!sed -i "s/CUDNN_HALF=0/CUDNN_HALF=1/" Makefile
```

備註：若以上指令無法更改可自colab左邊目錄直接打開 /content/darknet/Makefile檔案，進行修改。



Train yolov4-tiny

- 接著準備好訓練YOLOv4-tiny模型所需檔案，並上傳到雲端：
 - obj.zip (包含資料集影像與同名的.txt座標文件(使用LabelImg工具取得))
 - obj.names (自己資料集的類別名稱)
 - obj.data (相關路徑及各種參數設定)
 - Yolov4-tiny-custom.cfg (自己根據資料及類別數量修改)
 - generate_train.py (用來產生train.txt)





Train yolov4-tiny

- 上傳obj.data和obj.names到雲端：
 - obj.data 根據自己資料集的類別個數進行修改而得到。
 - obj.names中每行寫入自己資料集的類別名稱。

obj.data

*obj.data

```
1 classes=4
2 train=/content/drive/MyDrive/FinalProject/train.txt
3 valid=/content/drive/MyDrive/FinalProject/test.txt
4 names=/content/drive/MyDrive/FinalProject/obj.names
5 backup=/content/drive/MyDrive/FinalProject/backup
```

```
classes=4
train=/content/drive/MyDrive/FinalProject/train.txt
valid=/content/drive/MyDrive/FinalProject/test.txt
names=/content/drive/MyDrive/FinalProject/obj.names
backup=/content/drive/MyDrive/FinalProject/backup
```

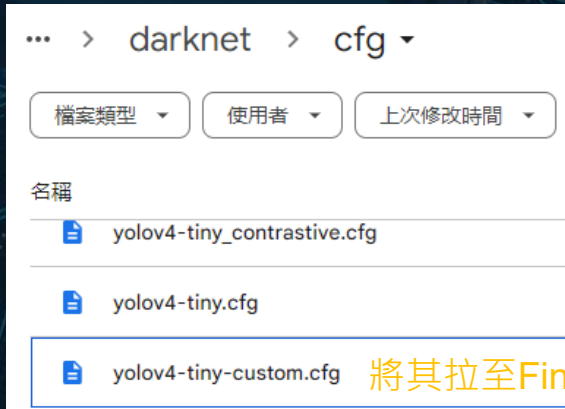
obj.names

```
1 stop
2 pedestrian
3 rail
4 blocked
```




Train yolov4-tiny

- 請參考 Project04 的投影片，此份投影片只會講解與Project04不同的地方。
- Project04中的yolov4_custom.cfg，須改下載 yolov4-**tiny**-custom.cfg，其路徑在 **darknet>cfg** 內



將其拉至FinalProject目錄下



Train yolov4-tiny

- 修改剛下載下來的yolov4-tiny-custom.cfg
- 修改一：
 - batch = 64, subdivisions = 16
 - 當訓練運行顯示“out of memory”時，可以嘗試將subdivisions升至32或64。
 - max_batches = ? 為自己資料集種類個數*2000，
 - 如果有一類即為2000
 - Steps = ?, ? 分別為max_batches的80%和90%

(可依據自己專案情況決定增加or減少訓練迭代次數)

yolov4-tiny-custom.cfg

```
1 [net]
2 # Testing
3 #batch=1
4 #subdivisions=1
5 # Training
6 batch=64
7 subdivisions=16
8 width=416
9 height=416
10 channels=3
11 momentum=0.9
12 decay=0.0005
13 angle=0
14 saturation = 1.5
15 exposure = 1.5
16 hue=.1
17
18 learning_rate=0.00261
19 burn_in=1000
20 max_batches = 8000
21 policy=steps
22 steps=6400, 7200
23 scales=.1, .1
```




Train yolov4-tiny

- 修改剛下載下來的yolov4-tiny-custom.cfg
- 修改二：
 - $\text{filters} = (5 + \text{classes}) \times 3$ 如資料集種類個數classes為4 則filters=27
 - 共要修改兩個部分，如下圖所示： 與yolov4不同，此專案只有兩個部份要修改，改完就可以上傳了

```
208 [convolutional]
209 size=1
210 stride=1
211 pad=1
212 filters=27
213 activation=linear
214
215
216
217 [yolo]
218 mask = 3, 4, 5
219 anchors = 10, 14, 23, 27, 37, 58,
220 classes=4
221 num=6
```

```
259 [convolutional]
260 size=1
261 stride=1
262 pad=1
263 filters=27
264 activation=linear
265
266 [yolo]
267 mask = 0, 1, 2
268 anchors = 10, 14, 23, 27,
269 classes=4
```

Train yolov4-tiny

- 上傳generate_train.py文件到雲端：
 - 此yolov4-tiny專案訓練自己資料集還需要train.txt，train.txt檔可通過運行generate_train.py生成，所以在此之前需要創建generate_train.py，並上傳到雲端中，以便後續複製到Colab中並運行。generate_train.py文件如下：

```
import os
image_files = []
# 進到 data/obj 資料夾
os.chdir(os.path.join("obj"))
print(os.getcwd())

# 把資料夾裡所有 .jpg 圖片的路徑存起來
for filename in os.listdir(os.getcwd()):
    if filename.endswith(".jpg"):
        image_files.append(os.path.join(os.getcwd(), filename))

# 回到上一層
os.chdir("..")

# 把路徑寫進 train.txt
with open("train.txt", "w") as outfile:
    for image in image_files:
        outfile.write(image)
        outfile.write("\n")

# 再回到專案根目錄 (如果需要)
os.chdir("..")
```

```
generate_train.py
1 import os
2 image_files = []
3 # 進到 data/obj 資料夾
4 os.chdir(os.path.join("obj"))
5 print(os.getcwd())
6
7 # 把資料夾裡所有 .jpg 圖片的路徑存起來
8 for filename in os.listdir(os.getcwd()):
9     if filename.endswith(".jpg"):
10         image_files.append(os.path.join(os.getcwd(), filename))
11
12 # 回到上一層
13 os.chdir("..")
14
15 # 把路徑寫進 train.txt
16 with open("train.txt", "w") as outfile:
17     for image in image_files:
18         outfile.write(image)
19         outfile.write("\n")
20
21 # 再回到專案根目錄 (如果需要)
22 os.chdir("..")
```

執行此 python 檔會根據 obj.zip 內檔案內容自動產生所有路徑的 train.txt 檔案



Train yolov4-tiny

- 將訓練所需檔案上傳至雲端後，依照右側指令依序copy至Colab工作空間，全部準備完成後就可以開始訓練自己的YOLOv4-tiny模型了。

```
%cd ..
```

```
/content/drive/MyDrive/FinalProject
```

```
!unzip obj.zip
```

顯示隱藏的輸出內容

```
!python generate_train.py
```

```
/content/drive/MyDrive/FinalProject/obj
```

```
%cd darknet/
```



Train yolov4-tiny

- Makefile修改完、generate_train.py執行完後，我們就用 make 指令來生成 darknet 這個深度學習引擎了。

```
!sudo apt update  
!sudo apt install -y libopencv-dev
```

```
▶ # 進行編譯  
!make clean # 若過程中有失敗，執行 make clean 再重新編譯可以避免掉某些未知錯誤  
!make
```

- 在 make 完成後，darknet 這套深度學習引擎就已經安裝完畢了，接下來我們會想要測試他是否能正常工作，所以這時候我們就先去下載一些已經預先 train 好的 weights 檔做為測試用。

```
▶ # 設定執行檔權限，這一步只有在自己雲端硬碟內執行darknet的情況下才需要  
!chmod 755 ./darknet
```




Train yolov4-tiny

- 下載yolov4-tiny預訓練權重和測試權重檔
- 下載測試權重檔指令：
- `!wget https://github.com/AlexeyAB/darknet/releases/download/yolov4/yolov4-tiny.weights`
- 下載預訓練權重檔指令：
- `!wget https://github.com/AlexeyAB/darknet/releases/download/yolov4/yolov4-tiny.conv.29`

```
[ ] 1 # 測試權重檔
    2 !wget https://github.com/AlexeyAB/darknet/releases/download/yolov4/yolov4-tiny.weights
    3
    4 # 預訓練權重檔
    5 !wget https://github.com/AlexeyAB/darknet/releases/download/yolov4/yolov4-tiny.conv.29
```



Train yolov4-tiny

- 下載完，應該會在darknet資料夾底下看到這兩個檔案。



yolov4-tiny.conv.29



yolov4-tiny.weights



Train yolov4-tiny

- 初次訓練

要先建立 backup 資料夾放在FinalProject下

- `!./darknet detector train data/obj.data cfg/yolov4-tiny-custom.cfg yolov4-tiny.conv.29 -dont_show`

```
!./darknet detector train data/obj.data cfg/yolov4-tiny-custom.cfg yolov4-tiny.conv.29 -dont_show
```

obj.data位置

cfg位置

預訓練銓重檔位置

- 備份權重檔

- `!cp ./backup/yolov4-tiny_custom_last.weights /mydrive/finalProject`

#備份權重檔

```
!cp ./backup/yolov4-tiny_custom_last.weights /mydrive/finalProject
```

- 接續訓練

- `!./darknet detector train data/obj.data cfg/yolov4-tiny-custom.cfg backup/yolov4-tiny_custom_last.weights -dont_show`

#繼續之前訓練到一半的模型

```
!./darknet detector train data/obj.data cfg/yolov4-tiny-custom.cfg backup/yolov4-tiny_custom_last.weights -dont_show
```



Jetbot安裝環境

- 回到小車上，安裝環境（在桌面右鍵選擇Open Terminal）
- `$ sudo pip install protobuf==3.8.0`

```
jetbot@jetson-4-3:~$ sudo pip install protobuf==3.8.0
Collecting protobuf==3.8.0
  Downloading https://files.pythonhosted.org/packages/8f/5a/077f2811869284d8e15fcc88818922e19cf205a3529f21210f3a1b47dc74/protobuf-3.8.0-py2.py3-none-any.whl (427kB)
    |████████████████████| 430kB 1.5MB/s
Requirement already satisfied: setuptools in /usr/local/lib/python3.6/dist-packages (from protobuf==3.8.0) (44.0.0)
Requirement already satisfied: six>=1.9 in /usr/lib/python3/dist-packages (from protobuf==3.8.0) (1.11.0)
Installing collected packages: protobuf
  Found existing installation: protobuf 3.11.2
  Uninstalling protobuf-3.11.2:
    Successfully uninstalled protobuf-3.11.2
Successfully installed protobuf-3.8.0
```




Jetbot安裝環境

```
$ sudo apt-get install protobuf-compiler libprotoc-dev
```

```
jetbot@jetson-4-3:~$ sudo apt-get install protobuf-compiler libprotoc-dev
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libprotobuf-dev libprotobuf-lite10 libprotobuf10 libprotoc10
The following NEW packages will be installed:
  libprotobuf-dev libprotoc-dev libprotoc10 protobuf-compiler
The following packages will be upgraded:
  libprotobuf-lite10 libprotobuf10
2 upgraded, 4 newly installed, 0 to remove and 648 not upgraded.
Need to get 2,917 kB of archives.
After this operation, 16.3 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://ports.ubuntu.com/ubuntu-ports bionic-updates/main arm64 libprotobuf-lite10 arm64 3.0.0-9.1ubuntu1.1 [93.6 kB]
```



Jetbot安裝環境

`$ sudo pip install onnx==1.4.1`

```
jetbot@jetson-4-3:~$ sudo pip install onnx==1.4.1
Collecting onnx==1.4.1
  Downloading onnx-1.4.1.tar.gz (2.9 MB)
    |████████████████████████████████████████| 2.9 MB 1.8 MB/s
    Preparing metadata (setup.py) ... done
```

`$ sudo pip install --upgrade pip`

```
jetbot@jetson-4-3:~$ sudo pip install --upgrade pip
Keyring is skipped due to an exception: org.freedesktop.
Collecting pip
  Downloading https://files.pythonhosted.org/packages/a4
    |████████████████████████████████████████| 1.7MB 1.7MB/s
Installing collected packages: pip
  Found existing installation: pip 19.3.1
  Uninstalling pip-19.3.1:
    Successfully uninstalled pip-19.3.1
  Successfully installed pip-21.3.1
```




Jetbot安裝環境

`$ sudo pip uninstall enum34`

```
jetbot@jetson-4-3:~$ sudo pip uninstall enum34
Keyring is skipped due to an exception: org.freedes
Found existing installation: enum34 1.1.6
Uninstalling enum34-1.1.6:
  Would remove:
    /usr/local/lib/python3.6/dist-packages/enum/*
    /usr/local/lib/python3.6/dist-packages/enum34-1
Proceed (Y/n)? Y
Successfully uninstalled enum34-1.1.6
```



Jetbot安裝環境

```
$ gedit ~/.bashrc
```

```
jetbot@jetson-4-3:~$ gedit ~/.bashrc
```

下完指令會跳出右側的視窗

在檔案的最下面加這兩行：

```
export PATH=/usr/local/cuda-10.2/bin:${PATH} (cuda版本請去確認)
```

```
Export LD_LIBRARY_PATH=/usr/local/cuda-10.2/lib64:${LD_LIBRARY_PATH}
```

加完存檔關閉bashrc，並且關掉重啟剛才的terminal。

接著在terminal輸入以下指令

```
$ export CUDA_ROOT=/usr/local/cuda
```

```
$ pip3 install pycuda --verbose
```

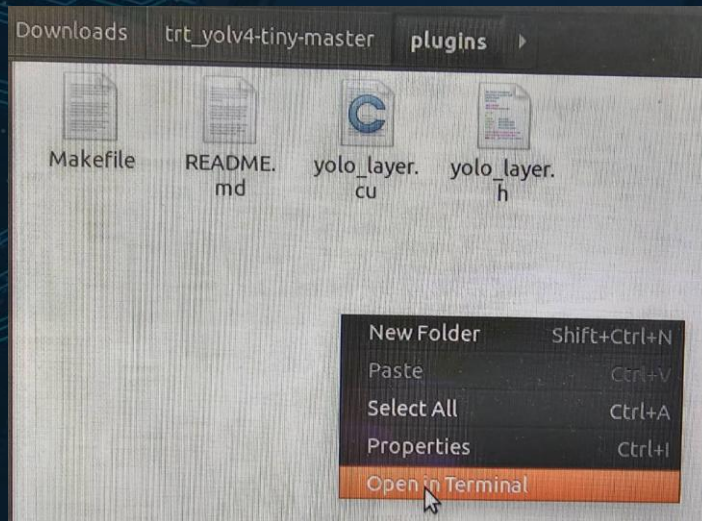
```
jetbot@nano-4gb-jp45:~$ ls /usr/local/  
bin  cuda-10.2  games  jetson_stats  lib  sbin  src  
cuda  etc      include_ jtop  man  share
```

```
*.bashrc (~) - gedit  
Open  Save  
jetbot  *.bashrc  
# colored GCC warnings and errors  
#export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01'  
# some more ls aliases  
alias ll='ls -aF'  
alias la='ls -A'  
alias l='ls -CF'  
# Add an "alert" alias for long running commands.  Use like so:  
# sleep 10; alert  
alias alert='notify-send --urgency=low -i "${?} = 0" && echo terminal || echo error)' "$  
(history|tail -n1|sed -e '\s/\s*[0-9]+\s*//;s/;/&])\s*alert$//'\''"  
# Alias definitions.  
# You may want to put all your additions into a separate file like  
# ~/.bash_aliases, instead of adding them here directly.  
# See /usr/share/doc/bash-doc/examples in the bash-doc package.  
if [ -f ~/.bash_aliases ]; then  
    . ~/.bash_aliases  
fi  
# enable programmable completion features (you don't need to enable  
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile  
# sources /etc/bash.bashrc).  
if ! shopt -oq posix; then  
    if [ -f /usr/share/bash-completion/bash_completion ]; then  
        . /usr/share/bash-completion/bash_completion  
    elif [ -f /etc/bash_completion ]; then  
        . /etc/bash_completion  
    fi  
fi  
export PATH=/usr/local/cuda-10.0/bin:${PATH}  
export LD_LIBRARY_PATH=/usr/local/cuda-10.0/lib64:${LD_LIBRARY_PATH}
```




訓練完的模型轉換成ONNX檔

- 下載[trt_yolov4-tiny-master.zip](#)，並解壓縮
- 進入plugins資料夾，並右鍵Open in Terminal



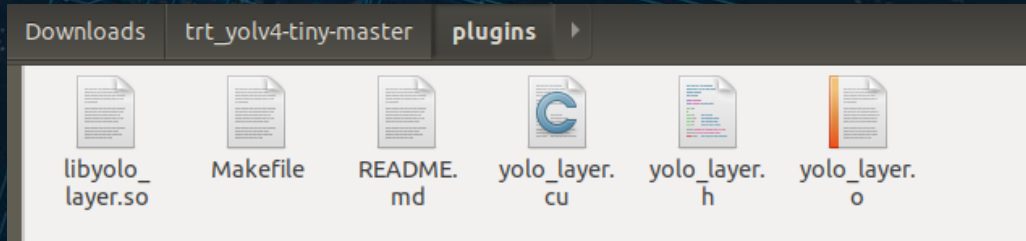


訓練完的模型轉換成ONNX檔

- 輸入 `make -j4`

```
jetbot@jetson-4-3:~/Downloads/trt_yolv4-tiny-master/plugins$ make -j4
g++ -shared -o libyolo_layer.so yolo_layer.o -L"/usr/local/cuda/lib64" -L"/usr/local/TensorRT-7.1.3.4/lib" -L"/usr/local/lib" -Wl,--start-group -lnvinfer -lnvpar
sers -lnvinfer_plugin -lcudnn -lcublas -lcudart_static -lnvToolsExt -lcudart -l
rt -ldl -lpthread -Wl,--end-group
```

- 輸入後會發現資料夾多一個檔案
`libyolo_layer.so`

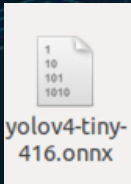




訓練完的模型轉換成ONNX檔

- 把colab訓練的cfg檔名改成yolov4-tiny-416.cfg
- 把colab訓練的weights檔名改成yolov4-tiny-416.weights放在yolo資料夾內
- 在trt_yolov4-tiny-master>yolo資料夾內按右鍵Open in terminal
- 輸入 `python3 yolo_to_onnx.py -c 4 -m yolov4-tiny-416`

```
jetbot@jetson-4-3:~/Downloads/trt_yolov4-tiny-master/yolo$ python3 yolo_to_onnx.py -c 4 -m yolov4-tiny-416
Checking ONNX model...
Saving ONNX file...
Done.
```



這步驟是把模型轉成onnx

輸入完會發現資料夾內多了一個onnx的檔案

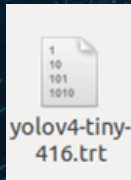


訓練完的模型轉換成ONNX檔

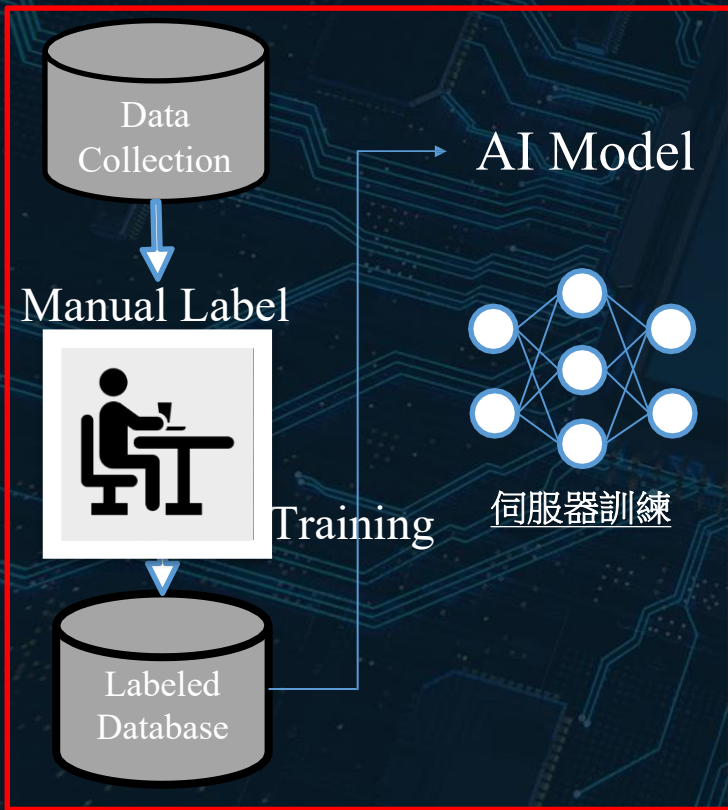
- 接著輸入 `python3 onnx_to_tensorrt.py -c 4 -m yolov4-tiny-416`

```
jetbot@jetson-4-3:~/Downloads/trt_yolov4-tiny-master/yolo$ python3 onnx_to_tensorrt.py -c 4 -m yolov4-tiny-416
Loading the ONNX file...
Adding yolo_layer plugins...
Building an engine. This would take a while...
(Use "--verbose" or "-v" to enable verbose logging.)
Completed creating engine.
Serialized the TensorRT engine to file: yolov4-tiny-416.trt
```

- 這步驟是把模型轉成trt，輸入完會發現資料夾內多了一個trt的檔案



模型訓練和部屬



手機



CPU



GPU



ASIC





模型訓練框架



伺服器訓練



AI Model



模型訓練和部屬

手機



CPU



GPU



ASIC



1. 模型怎麼在不同的運算機器上跑推論
OS: Windows, Linux, RTOS

AI engine: Pytorch, Tensforflow, Caffe, Mxnet, ... Inference NN accelerator: CMSIS-NN, TensorFlow Lite, OpenVINO, TensorRT, Onnxruntime, ...

2. 不同的訓練架構要怎麼互通



3. 是否有相同的運算架構做這樣的事情





PyTorch

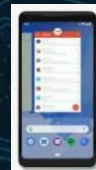


K Keras

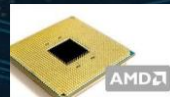


表示更容易部署的靜態圖

手機



CPU



GPU



ASIC

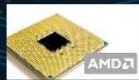




手機



CPU



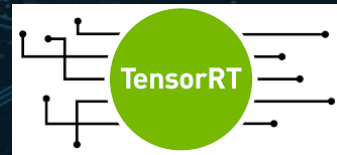
GPU



ASIC



FP-64/32/16
INT-
16/8/4/1





ONNX 簡介



- 神經網路框架格式間的互通性，讓不同標準的深度學習模型可以互相轉換，加快AI人工智慧的發展。
- ONNX: Open Neural Network Exchange: 開放神經網路交換格式。
- 由Facebook、微軟結盟推出
- 並獲得 Intel、AMD、ARM、IBM、華為和高通/聯發科晶片廠商支持加入。→大部分SoC都有Support。
- 目前官方支持加載ONNX模型並進行推理的深度學習框架有：Caffe2, PyTorch, MXNet, ML.NET, TensorRT 和 Microsoft CNTK，並且 TensorFlow 也非官方的支持ONNX。



為甚麼要轉成ONNX



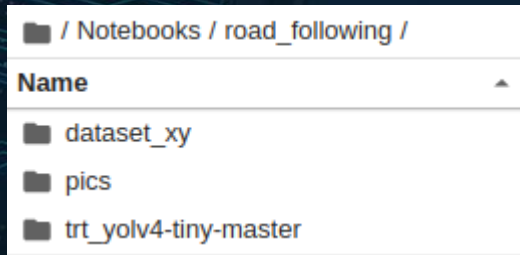
1. ONNX 讓模型變成 TensorRT 看得懂的格式
 - .weights 只有參數，沒有架構
 - TensorRT 無法解析 YOLO 原生格式
 - ONNX 提供完整計算圖（架構+流程+權重）沒有 ONNX，TensorRT 就無法開始工作

2. ONNX 為 TensorRT 建立固定的靜態計算圖
 - 明確定義每一層與運算順序
 - 方便 TensorRT 分析網路拓撲
 - 為後續最佳化提供藍圖ONNX = TensorRT 最佳化的基礎



偵測交通號誌執行動作

- 把trt_yolv4-tiny-master資料夾複製到road_following底下



- 並且在Notebooks>road_following>trt_yolv4-tiny-master底下新增一個Project06. ipynb

Project06. ipynb 範例：<https://colab.research.google.com/drive/1npnIN2KBDE6Vn-pR8UGVJU0xjw89Uri?usp=sharing>



偵測交通號誌執行動作

讀取TRT模型

```
[1]: import os
import time

import pycuda.autoninit
from utils.yolo_classes import get_cls_dict
from utils.display import open_window, set_display, show_fps
from utils.visualization import BBoxVisualization
from utils.yolo import TRT_YOLO

trt_yolo = TRT_YOLO("yolov4-tiny-416", (416, 416), 4)
```



偵測交通號誌執行動作

```
import cv2
import ipywidgets.widgets as widgets
from IPython.display import display

# 1. 讀取影像
img = cv2.imread('1.jpg') 可更換成自己拍的照片去讓模型辨識

# 2. 進行模型辨識
boxes, confs, cls = trt_yolo.detect(img, 0.3)

# 3. 跑迴圈將辨識結果畫在影像上
for i in range(len(cls)):
    cv2.rectangle(img, (boxes[i][0], boxes[i][1]), (boxes[i][2], boxes[i][3]), (255, 0, 0), 2)

# 將 OpenCV 的影像編碼成 JPEG 格式
_, encoded_image = cv2.imencode('.jpg', img)

# 建立一個 Image 元件並顯示
image_widget = widgets.Image(value=encoded_image.tobytes(), format='jpg', width=400)
display(image_widget)
```





偵測交通號誌執行動作

Import自走車套件

```
[3]: import traitlets
from IPython.display import display
import ipywidgets.widgets as widgets
from jetcam.usb_camera import USBCamera
from jetbot import Camera, bgr8_to_jpeg
from jetbot import Robot

camera = USBCamera(capture_device=0)
camera.running=True

# camera = Camera.instance(width=416, height=416)
image = widgets.Image(format='jpeg', width=224, height=224)
camera_link = traitlets.dlink((camera, 'value'), (image, 'value'), transform=bgr8_to_jpeg)
robot = Robot()
```

只有用USBCamera的需要這幾行

一般camera的用這一行

Error loading module `ublox\_gps`: No module named 'serial'

出現這個錯誤不影響



偵測交通號誌執行動作

- 小車走直線算法
- 檢測和跟隨道路線，保持直線行駛

走直線演算法

```
import numpy as np

def getROI(img, vertices):
    mask = np.zeros_like(img) 建立遮罩
    cv2.fillPoly(mask, vertices, (255,)) 把多邊形以外的區域篩掉
    masked_image = cv2.bitwise_and(img, mask)
    return masked_image 找要跟蹤的點

def make_points(image, average):
    slope, y_int = average
    y1 = image.shape[0]
    y2 = int(y1 * (1/6))
    x1 = int((y1 - y_int) // slope)
    x2 = int((y2 - y_int) // slope)
    return np.array([x1, y1, x2, y2])
```




偵測交通號誌執行動作

- 小車走直線算法
- 畫出行走路線

```
def average(image, lines):  
    if lines is None: 將多條線中畫出左右各一條線  
        return  
    left = []  
    right = []  
    for line in lines:  
        x1, y1, x2, y2 = line.reshape(4)  
        parameters = np.polyfit((x1, x2), (y1, y2), 1)  
        slope = parameters[0]  
        y_int = parameters[1]  
        if slope < 0:  
            left.append((slope, y_int))  
        else:  
            right.append((slope, y_int))  
  
    right_avg = np.average(right, axis=0)  
    left_avg = np.average(left, axis=0)  
    if type(left_avg) is np.float64:  
        right_line = make_points(image, right_avg)  
        return np.array([right_line,])  
    if type(right_avg) is np.float64:  
        left_line = make_points(image, left_avg)  
        return np.array([left_line,])  
  
    right_line = make_points(image, right_avg)  
    left_line = make_points(image, left_avg)  
    return np.array([left_line, right_line])
```



偵測交通號誌執行動作

- 小車走直線算法
- (可以根據之前車道線作業做調整)

```
def getAverageLines(origin_road):  
    origin_road = cv2.resize(origin_road, (640, 480))  
    road_gray = cv2.cvtColor(origin_road, cv2.COLOR_BGR2GRAY) 讀入RGB影像轉換為灰階影像  
    road_gray = cv2.GaussianBlur(road_gray, (5,5), 0) 濾波  
    edges = cv2.Canny(road_gray, 3000, 2000, apertureSize=5) 邊緣檢測  
  
    ROI_vertices = [(0, 480), (0, 150), (250, 80), (390, 80), (640, 150), (640, 480)]  
    edges = getROI(edges, np.array([ROI_vertices], np.int32))  
  
    #找直線  
    linesp = cv2.HoughLinesP(edges, rho=2, theta=np.pi / 180 直線偵測  
        , threshold=100, minLineLength=40, maxLineGap=5)  
  
    return average(origin_road, linesp)
```




偵測交通號誌執行動作

例如影像的寬度是 640 像素：
影像中央的 x 座標大約是 320。
315 和 325 就是圍繞中央位置
設定的一個小範圍，
用來判斷車道線是否偏離中央。

- mid_x 的值：
 - 這代表車道線平均位置的 x 座標（即影像中車道線的中間位置）。
 - 根據這個值，可以判斷車子是否偏離道路中央。
- 兩個情況判斷：
 - 當 mid_x < 315：車子偏右。
 - 動作：增加左輪速度 (speed_left += 0.05)
 - 稍微增加右輪速度 (speed_right += 0.03)
 - 這樣的調整使得車子向左修正。
 - 當 mid_x > 325：車子偏左。
 - 動作：增加右輪速度 (speed_right += 0.05)
 - 稍微增加左輪速度 (speed_left += 0.02)
 - 這樣的調整使得車子向右修正。
 - mid_x 介於 315 和 325 之間：車子在中央位置。
 - 動作：兩側輪速稍微增加 (speed_left += 0.01, speed_right += 0.01)
 - 這樣保持車子的直線前進。

```
def modifySpeed(origin_road):  
    global speed_left, speed_right, speed  
  
    averaged_lines = getAverageLines(origin_road)  
  
    if averaged_lines is not None:  
        mid_x = int(np.average(averaged_lines, axis=0)[2])  
  
        if mid_x < 315:  
            speed_left = speed  
            speed_right = speed + 0.03  
        elif mid_x > 325:  
            speed_left = speed + 0.02  
            speed_right = speed  
        else:  
            speed_left = speed + 0.01  
            speed_right = speed
```

需要根據
自己組別的車況
進行調整



偵測交通號誌執行動作

- 行車行為&速度演算法
- 偵測交通標誌並執行相應行車行為
- 根據不同的交通標誌來調整車輛行為或速度

行車行為 & 速度演算法

```
import time
stop_ignore = 0
railway_ignore = 0
count = 0

def getNearest(signs):
    if not len(signs):
        return [0, 3]

    t = time.time()

    if (signs[0][1] == 0 and (t > stop_ignore)) \
        or (signs[0][1] == 1 and (t > railway_ignore)) \
        or (signs[0][1] in [2, 3]):
        return signs[0]

    return getNearest(signs[1:]) if len(signs) >= 2 else [0, 3]
```




偵測交通號誌執行動作

- 行車行為&速度算法
- 可參考右側程式做調整，
- 減速行駛若減速的太多，
- 可能會導致車子卡住

偵測交通號誌需執行的動作：

當心行人：偵測到此號誌時，減速後再繼續行駛。

道路封閉：偵測到此號誌時停止Jetbot，停止位置不得超過交通號誌

停車再開：偵測到此號誌時，於原地停止2秒後再繼續行駛。

鐵路平交道：偵測到此號誌時，於原地停止5秒後再繼續行駛。

```
def update(change):
    global robot, count, speed_left, speed_right, stop_ignore, railway_ignore
    ALERT_WIDTH = 50

    img = change['new']
    modifySpeed(img)

    if count < 10:
        count += 1
        return
    else:
        count = 0

    boxes, confs, cls = trt_yolo.detect(img)

    signs = []
    for box, cls in zip(boxes, cls):
        width = box[2] - box[0]
        signs.append([width, cls])
    signs.sort(reverse=True, key=lambda x : x[0])
    sign = getNearest(signs)

    t = time.time()

    if sign[1] == 2 and sign[0] > ALERT_WIDTH: #減速行駛
        print('slow')
        robot.set_motors(speed_left * 0.7, speed_right * 0.7)

    if (sign[1] == 3 and sign[0] > ALERT_WIDTH): #道路封閉
        print("road close")
        robot.stop()

    elif sign[1] == 0 and sign[0] > ALERT_WIDTH: #停車再開
        print("stop")
        robot.stop()
        stop_ignore = t + 10
        time.sleep(3)

    elif sign[1] == 1 and sign[0] > (ALERT_WIDTH - 20): #鐵路平交道
        print("railway")
        robot.stop()
        railway_ignore = t + 10
        time.sleep(5)

    else:
        robot.set_motors(speed_left, speed_right)
```

偵測路牌

找出最靠近車子的路牌



偵測交通號誌執行動作

- 驗證鏡頭畫面是否正常
- 若想看到即時的鏡頭畫面
- 可以再新增一行

`display(image)`

```
import cv2
import ipywidgets.widgets as widgets
from IPython.display import display
import numpy as np

# 1. 驗證鏡頭畫面
img = camera.value

# 2. 偵測路牌標誌
boxes, confs, cls = trt_yolo.detect(img, 0.3)

# 3. 取得車道平均線
averaged_lines = getAveragedLines(img)

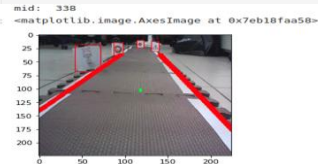
if averaged_lines is not None:
    for x1, y1, x2, y2 in averaged_lines:
        # 畫出偵測到的車道紅線 (座標縮放: 640->224, 480->224)
        cv2.line(img, (int(x1 / 640 * 224), int(y1 / 480 * 224)),
                  (int(x2 / 640 * 224), int(y2 / 480 * 224)), (0, 0, 255), 5)

    # 計算並畫出路徑中點的綠色標記
    mid_x = int(np.average(averaged_lines, axis=0)[2])
    cv2.line(img, (int(mid_x / 640 * 224), 100), (int(mid_x / 640 * 224), 105), (0, 255, 0), 2)
    print('mid: ', mid_x)

# 4. 畫出路牌的辨識框
for i in range(len(cls)):
    cv2.rectangle(img, (boxes[i][0], boxes[i][1]), (boxes[i][2], boxes[i][3]), (0, 0, 255), 1)

# 5. 【替換掉 imshow】使用 ipywidgets 顯示
# 因為 widgets.Image 會直接處理編碼，這裡不需要手動將 BGR 轉 RGB 也能正確顯示
_, encoded_image = cv2.imencode('.jpg', img)
image_widget = widgets.Image(value=encoded_image.tobytes(), format='jpg', width=224, height=224)

display(image_widget)
```





偵測交通號誌執行動作

- 設定車子的行走速度
範例車右邊馬達比較弱，
所以右邊speed設比較高。
可根據車子自身條件調整。

給左右馬達速度

```
speed = speed_left = speed_right = 0.35
```

```
robot.set_motors(0.37, 0.41)
```

 車子起步時的速度

```
time.sleep(0.3)
```

```
robot.set_motors(speed_left, speed_right)
```

```
while(1):
```

```
    update({'new' : camera.value})
```

 設定速度

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
/usr/local/lib/python3.6/dist-packages/ipykernel
```

```
road close
```

 偵測到的結果



偵測交通號誌執行動作

- 想要車子停下來，要先把這段stop

```
speed = speed_left = speed_right = 0.35  
  
robot.set_motors(0.37, 0.41)  
time.sleep(0.3)  
robot.set_motors(speed_left, speed_right)  
  
while(1):  
    update({'new' : camera.value})
```

- 再執行這一段

```
speed = speed_left = speed_right = 0  
robot.stop()
```




偵測交通號誌執行動作

- Unlink camera 中止瀏覽器跟camera的連結(但camera仍會運作)

```
camera_link.unlink() # don't stream to browser (will still run camera)
```

- 停止camera運作

```
camera.stop()
```

USBCamera不需要執行此行

Project 6

Jetbot路標辨識

作業檢查時間: 115/6/10

小組報告繳交期限: 115/6/12 23:59

個人報告繳交期限: 115/6/12 23:59



專案實作-路標辨識

- 專案項目: Jetbot路標辨識
- 使Jetbot能夠偵測交通號誌執行動作。
- 本專案應於Jetbot實作。(70%)
 - 收集資料。(10%)
 - 訓練模型。(20%)
 - Jetbot路標偵測與走直線。(40%)
- 撰寫操作過程學習心得並以小組報告(PDF)、實作code、個人報告(PDF)、影片連結。(30%)



小組報告格式規定

- 實驗內容
 - 此次實驗需要做什麼？
- 實驗過程及結果
 - 預期實驗的結果
 - 實際上的結果
 - 遇到的問題&問題怎麼解決？
- 本次實驗過程說明與解決方法(為上述”實驗過程及結果”的摘要、總結)
 - 實驗過程
 - 解決方法
- 成果展示
 - 影片展示
- 分工
 - 說明小組成員分工內容與比例。



個人報告內容

- 實驗心得：（150字）
 - 此次實驗的目的？
 - 在實驗上遇到的問題？怎麼解決？
 - 實驗結果跟你預期的內容是否一致？
- 組員貢獻度及工作內容：
 - 陳大明：33 %，資料查詢、文書處理、實驗實作。
 - 陳忠義：33 %，實驗設計、程式規劃、測試與除錯。
 - 陳小美：33 %，小組報告、買飲料、買午餐。



專案繳交規則

- 作業檢查時間:115/6/10
- 小組報告繳交期限:115/6/12 23:59
- 個人報告繳交期限:115/6/12 23:59(以I學園上傳時間為基準)
- 補交規則
 - 超過正常繳交期限一周成績**打9折**，每再遲交一週成績**打8折**(最多遲交兩週)



Project Demo



Demo1

<https://youtube.com/shorts/R6niYaoVEP0?feature=share>



Demo2

https://youtube.com/shorts/faCIM7swb_k?feature=share